

1 INTRODUCTION / GENERAL QUESTIONS

(Speak calmly, 60–90 seconds max for each)

✓ Walk me through your resume

Answer:

I am Aditya Maurya, currently pursuing **MCA from MNNIT Allahabad** with a CPI of **7.82**. I completed my **BSc** in 2023, where I built a strong foundation in computer science.

During MCA, I worked on two major projects.

The first is an **Automatic Attendance System**, which uses **face recognition with Raspberry Pi**, OpenCV, and dlib, achieving **95% accuracy** in offline mode.

My second project is **DailyFit**, a **MERN stack** platform that allows users to book one-day gym passes and helps gym owners manage slots and pricing.

I have solved **950+ DSA problems** across platforms like LeetCode and GFG and secured **AIR 249 in NIMCET**.

Along with academics, I have been active in the **Application Club** and sports, which helped me develop teamwork and leadership skills.

✓ Tell me about yourself

Answer:

I am a motivated MCA student who enjoys solving problems and building real-world applications. I have a strong interest in **data structures, backend development, and applied machine learning**.

I like projects that solve practical problems, such as attendance automation and flexible fitness access.

I believe in continuous learning, consistency, and teamwork, which aligns well with a company like TCS.

✓ Why did you choose MCA after BSc?

Answer:

During my BSc, I realized that I enjoyed **programming and problem-solving more than theory**. MCA allowed me to gain **in-depth technical knowledge**, industry-oriented skills, and hands-on project experience.

It helped me move from basic concepts to **system design, DSA, and full-stack development**, which prepared me for software engineering roles.

✓ Why MCA from MNNIT Allahabad?

Answer:

MNNIT Allahabad has a strong academic structure, competitive environment, and excellent coding culture.

The curriculum focuses on **core CS subjects, projects, and problem-solving**, which helped me grow technically.

Peer learning and exposure here pushed me to improve my consistency and discipline.

 What have you learned during your MCA so far?

Answer:

I have learned **strong fundamentals of DSA, DBMS, OS, and OOPS**, along with practical implementation through projects.

I also learned how to design scalable applications, debug efficiently, and write clean code.

Most importantly, I learned **how to learn quickly**, which is crucial in the IT industry.

 How is your academic performance (CPI 7.82)?

Answer:

My CPI is **7.82**, which reflects consistent performance along with active involvement in projects, coding contests, and problem-solving.

I focused more on **practical learning and skill development**, which helped me perform well in competitive exams and interviews.

 What are your strengths and weaknesses?

Answer:

Strengths:

- Strong problem-solving skills
- Consistent practice in DSA
- Ability to learn new technologies quickly

Weakness:

- Earlier, I used to spend too much time perfecting solutions
 - Now, I focus on **balanced optimization and timely delivery**
-

 Why should TCS hire you for the Prime role?

Answer:

TCS Prime looks for **strong problem solvers with learning ability**, and I fit well into that role.

My experience with **DSA, real-world projects, ML, and full-stack development** shows my versatility.

I am disciplined, adaptable, and ready to contribute to complex systems at scale, which aligns with TCS's long-term vision.

 Where do you see yourself in 5 years?

Answer:

In 5 years, I see myself as a **technically strong software engineer** handling critical systems and mentoring juniors.

I want to grow within TCS by continuously upgrading my skills and contributing to impactful projects.

 Are you comfortable with relocation and night shifts?

Answer:

Yes, I am completely comfortable with relocation and night shifts.

I understand that global projects require flexibility, and I am prepared for that.

 DSA & CODING (PRIME ROLE FOCUS)

◆ **DSA CONCEPTS**

 Time complexity of binary search

Binary search works by dividing the search space into half each time.

Time complexity is **$O(\log n)$** .

 Array vs Linked List

Array

Linked List

Fixed size

Dynamic size

Fast access ($O(1)$)

Slow access ($O(n)$)

Contiguous memory

Non-contiguous memory

 Stack vs Queue (real use cases)

Stack (LIFO):

- Function calls
- Undo/Redo operations

Queue (FIFO):

- CPU scheduling
 - Print queue
-

✓ Hashing and collision handling

Hashing maps keys to index using a hash function.

If two keys map to the same index, it causes a **collision**.

Collision handling techniques:

- Chaining
 - Open addressing (linear/quadratic probing)
-

✓ BFS vs DFS

BFS:

- Uses queue
- Finds shortest path
- Level-wise traversal

DFS:

- Uses stack/recursion
 - Goes deep first
 - Less memory in some cases
-

✓ Recursion vs Iteration

Use **recursion** when the problem has a recursive structure like trees.

Use **iteration** when performance and stack memory are important.

◆ CODING QUESTIONS (C++)

✓ Reverse a linked list

```
ListNode* reverseList(ListNode* head) {  
    ListNode* prev = NULL;  
    while (head) {  
        ListNode* next = head->next;  
        head->next = prev;  
        prev = head;  
        head = next;  
    }  
    return prev;  
}
```

✅ Find duplicate elements in an array

```
int findDuplicate(vector<int>& nums) {  
    unordered_set<int> s;  
    for (int x : nums) {  
        if (s.count(x)) return x;  
        s.insert(x);  
    }  
    return -1;  
}
```

✅ Check if string is palindrome

```
bool isPalindrome(string s) {  
    int l = 0, r = s.size() - 1;  
    while (l < r) {  
        if (s[l++] != s[r--]) return false;  
    }  
    return true;  
}
```

✅ Two Sum approach

```
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int,int> mp;
    for (int i = 0; i < nums.size(); i++) {
        int rem = target - nums[i];
        if (mp.count(rem)) return {mp[rem], i};
        mp[nums[i]] = i;
    }
    return {};
}
```

✓ Find middle of linked list

```
ListNode* middleNode(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

✓ Detect cycle in linked list

```
bool hasCycle(ListNode *head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```

CORE CS (LIGHT BUT EXPECTED)

DBMS

What is normalization?

Normalization is the process of organizing data to **reduce redundancy** and **avoid data anomalies** by dividing tables into smaller related tables.

Difference between SQL and NoSQL

SQL databases are relational, table-based, and follow a fixed schema.

NoSQL databases are flexible, schema-less, and suitable for large-scale and unstructured data.

What is indexing?

Indexing is a technique used to **speed up data retrieval** by creating a reference structure on columns, similar to an index in a book.

ACID properties

Atomicity – Transaction is all or nothing

Consistency – Data remains valid

Isolation – Transactions don't interfere

Durability – Data persists after commit

OPERATING SYSTEM

Process vs Thread

A **process** is an independent program with its own memory.

A **thread** is a lightweight unit within a process that shares memory.

What is deadlock?

Deadlock is a situation where **two or more processes wait indefinitely** for resources held by each other.

Paging vs Segmentation

Paging divides memory into fixed-size blocks.

Segmentation divides memory based on logical units like functions or variables.

OOPS

✓ What is abstraction?

Abstraction means **showing only essential features** and hiding implementation details.

✓ Inheritance vs Composition

Inheritance represents an “is-a” relationship.

Composition represents a “has-a” relationship and is more flexible.

✓ Polymorphism with example

Polymorphism allows the **same function name to behave differently**.

Example: Function overloading or method overriding.

ACHIEVEMENTS & EXTRA CURRICULAR

✓ What did you learn from solving 950+ DSA problems?

I developed **strong problem-solving skills**, learned multiple approaches for the same problem, and improved my ability to write optimized code under constraints.

✓ Explain your experience in Application Club

I was part of the **question testing team**, where I reviewed coding problems, verified edge cases, and ensured clarity before contests.

✓ What was your role as Student Sports Coordinator?

I handled **event planning, coordination, and execution**, ensuring smooth communication between teams and management.

✓ How does cricket help you professionally?

Cricket taught me **teamwork, discipline, handling pressure**, and staying calm during critical situations.

✓ How do you manage time between academics and coding?

I follow **daily planning**, set clear priorities, and maintain consistency instead of last-minute preparation.

7 HR + BEHAVIORAL (VERY IMPORTANT)

✓ Why TCS?

TCS offers **long-term growth, learning opportunities, and exposure to global projects**, which aligns with my career goals.

✓ Why TCS Prime specifically?

TCS Prime focuses on **strong technical fundamentals, problem-solving, and complex systems**, which matches my skill set and learning mindset.

✓ Difference between TCS Ninja, Digital, and Prime

Ninja – Entry-level role

Digital – Higher technical exposure

Prime – Advanced problem-solving and critical systems

✓ How do you handle failure?

I analyze what went wrong, learn from it, and improve my approach instead of repeating the same mistake.

✓ Describe a challenging situation and how you handled it

While working on my attendance project, lighting variations reduced accuracy.
I solved it by improving dataset diversity and preprocessing, which increased accuracy to 95%.

✓ Are you ready for long-term commitment with TCS?

Yes. I am looking for **long-term growth**, and TCS provides the right environment for it.

✓ Any questions for us?

Best options (ask only ONE):

- What learning opportunities are provided for Prime engineers?
- How does TCS support career growth in the initial years?

2 PROJECT 1: AUTOMATIC ATTENDANCE SYSTEM

(MOST IMPORTANT FOR TCS PRIME)

◆ CORE UNDERSTANDING

✓ Explain your Automatic Attendance System end-to-end

Answer:

My project is an **ML-powered offline automatic attendance system** deployed on a **Raspberry Pi**.

The system captures live video using a camera, detects faces using OpenCV, and recognizes them using **dlib's 128-D facial embeddings**.

During enrollment, facial encodings are generated and stored locally.

During attendance, live face encodings are compared with stored encodings using distance thresholding.

If a match is found, attendance is marked, a **voice announcement** is made using pyttsx3, and the record is saved locally.

The system works **fully offline**, ensures data privacy, and automatically deletes temporary images to manage storage.

✓ Why did you choose this project?

Attendance marking is time-consuming and error-prone when done manually.

I wanted to build a **contactless, automated, and offline solution** that can work even in low-infrastructure environments like classrooms.

✓ What real-world problem does it solve?

It eliminates proxy attendance, reduces manual effort, saves time, and ensures accurate attendance without internet dependency.

✓ Why face recognition instead of fingerprint / RFID?

Fingerprint requires physical contact and fails if sensors are damaged.

RFID cards can be shared.

Face recognition is contactless, harder to misuse, and faster, making it suitable for classrooms.

✓ Why did you use Raspberry Pi?

Raspberry Pi is **cost-effective, portable, and energy efficient**.

It is ideal for deploying lightweight ML models at the edge without needing cloud infrastructure.

✓ Why Python for this project?

Python has excellent support for **OpenCV**, **dlib**, **NumPy**, and hardware integration, and it allows rapid development and easy debugging.

◆ ML / COMPUTER VISION

✓ Difference between face detection and face recognition

Face detection identifies *where* a face is in an image.

Face recognition identifies *whose* face it is by comparing features.

✓ What is a 128-D facial feature vector?

It is a numerical representation of a face consisting of **128 values** that uniquely describe facial features like distance between eyes, jaw shape, etc.

✓ How does dlib generate facial embeddings?

Dlib uses a **deep CNN model** trained on thousands of faces to convert a detected face into a **128-D embedding**, where similar faces have smaller Euclidean distance.

✓ Why did you use dlib over Haar Cascade only?

Haar Cascade is good for **face detection**, but not reliable for recognition.

Dlib provides **robust face recognition** with higher accuracy, especially under variations in lighting and angle.

✓ How did you achieve 95% accuracy?

I improved accuracy by:

- Using multiple images per person
- Handling lighting variations
- Applying proper face alignment
- Using optimized distance thresholds

I also tested with real classroom conditions instead of ideal images.

✔ **What dataset did you use? Did you create your own?**

I created my **own dataset** by capturing multiple images per user under different lighting and angles to make the system realistic.

✔ **How does lighting affect recognition and how did you handle it?**

Poor lighting affects feature extraction.

I handled it by:

- Capturing images under varied lighting
- Converting images to grayscale when required
- Using dlib's robust embeddings

✔ **What happens if two faces look similar?**

The system uses **distance thresholding**.

If distance is above the threshold, it is treated as an unknown face, preventing false attendance.

◆ **SYSTEM DESIGN & PERFORMANCE**

✔ **How does your system work offline?**

All facial data, encodings, and attendance records are stored **locally**.
No internet or cloud dependency is required.

✔ **How is data privacy ensured?**

Data never leaves the device.
No cloud upload or third-party API is used, ensuring **full privacy**.

✔ **Why local storage instead of cloud?**

Cloud increases cost, latency, and privacy risks.

Local storage ensures **low latency, offline reliability, and better control**.

✔ **What is the role of NumPy in your project?**

NumPy is used for:

- Handling facial embedding arrays
 - Fast mathematical operations
 - Distance calculations between face vectors
-

✓ **How does automated image deletion help?**

It prevents storage overflow, keeps the system fast, and avoids unnecessary data accumulation.

✓ **How scalable is this system for large classrooms?**

It works well for **small to medium classrooms**.

For large classrooms, optimization and server integration would be required.

◆ **IMPROVEMENTS / SCENARIO QUESTIONS**

✓ **What problems did you face while building this project?**

Lighting variations, false positives, and limited Raspberry Pi resources were major challenges.

The biggest challenge was arranging Raspberry Pi components during the Mahakumbh period in Prayagraj, when online deliveries were delayed. Because of this, I personally went to local electronics markets to search for compatible components like power supply, camera module, and connectors.

It took time and effort to verify compatibility and quality, but this helped me understand the hardware better. Finally, I assembled everything successfully and completed the project on time.

✓ **How would you improve accuracy further?**

By using:

- Better cameras
 - Larger datasets
 - Face alignment optimization
 - Advanced deep learning models
-

✓ **How would you integrate this system with a central server?**

The Raspberry Pi can send attendance data periodically using **REST APIs** to a central server for analytics and reporting.

✅ **Can this system work in real-time for 100+ students? How?**

Yes, by:

- Using batch processing
- Optimizing frame rate
- Using a stronger edge device or GPU-enabled server

🔗 **Why I Chose Dlib**

1. **Lightweight & CPU Friendly**
→ Works well on **Raspberry Pi (no GPU needed)**
2. **High Accuracy**
→ Uses **128-D face embeddings**, very reliable
3. **Offline Processing**
→ No internet required (important for attendance systems)
4. **Simple Integration**
→ Easy to use with **Python + OpenCV**
5. **Stable & Proven**
→ Widely used in **academic and real projects**

🧠 **One-line Answer (Best):**

"I chose Dlib because it gives good accuracy, works offline, and runs smoothly on Raspberry Pi without GPU."

MTCNN / FaceNet / ArcFace

- **MTCNN** – deep face detection network.
- **FaceNet / ArcFace** – produce *embeddings* (vectors) for recognition.
- Often more accurate than Dlib's face embeddings.
- Usually implemented as **TensorFlow/PyTorch models**.
- **May be heavy for Pi CPU** without acceleration. [Reddit](#)

3 **PROJECT 2: DailyFit (MERN Stack)**

(Web + Real-Life Problem – Very Important for TCS)

◆ CORE QUESTIONS

✓ Explain the DailyFit project

Answer:

DailyFit is a **MERN stack web platform** that connects **users who want short-term gym access** with nearby gyms.

Users can sign up, log in, browse gyms, select a **date and time slot**, and purchase **one-day gym passes**.

Gym owners have a separate dashboard where they can **manage gym details, pricing, slots, and check-ins**.

The platform solves the problem of **long-term gym memberships** by offering flexible access.

✓ What problem does DailyFit solve?

Many users want to go to the gym occasionally but are forced to buy long-term memberships.

DailyFit provides **flexible, pay-per-use gym access**, benefiting both users and gym owners.

✓ Why MERN stack for this project?

MERN allows **fast development, scalability, and clean separation** between frontend and backend.

React handles dynamic UI, Node and Express handle APIs, and MongoDB provides flexible data storage — ideal for a growing platform.

✓ Why MongoDB instead of MySQL?

MongoDB is **schema-flexible**, which helps when gym data, pricing, and slots change frequently.

It also integrates naturally with Node.js and scales well for high read/write operations.

✓ What is the role of React in your project?

React handles:

- User and gym owner dashboards
- Real-time slot selection
- Dynamic UI updates without page reload

It improves user experience and performance.

◆ BACKEND / DATABASE

✅ How does user authentication work?

Authentication is implemented using:

- Email and password
- Password hashing (bcrypt)
- JWT tokens for session management

Tokens are verified on protected routes to ensure secure access.

✅ How do you manage booking availability?

Each gym has predefined **time slots**.

Before booking, the system checks:

- Slot availability
- Date and time conflicts

If available, the slot is reserved and updated in the database.

✅ How do gym owners manage slots and pricing?

Gym owners have a dashboard where they can:

- Create or update slots
- Change pricing
- View bookings and check-ins

All changes are reflected in real time for users.

✅ How is data stored in MongoDB (collections)?

Major collections include:

- Users
- GymOwners
- Gyms
- Bookings
- Payments

Each collection is linked using ObjectIds for efficient referencing.

✓ How do you handle concurrent bookings?

Before confirming a booking, the backend:

- Rechecks slot availability
- Locks the slot during payment

This prevents double booking.

◆ SECURITY & SCALABILITY

✓ How do you secure user data?

Security measures include:

- Password hashing
- JWT authentication
- Role-based access (user vs owner)
- Input validation

Sensitive data is never exposed on the frontend.

✓ What happens if payment fails?

If payment fails:

- Booking is not confirmed
- Slot is released
- User receives failure status

This ensures data consistency.

✓ How will you scale this platform to multiple cities?

Scaling can be done by:

- Adding location-based gym filtering
- Using indexes on location fields
- Deploying backend on cloud with load balancing

MongoDB supports horizontal scaling.

✅ How will you optimize performance?

Performance optimization includes:

- Database indexing
- API response optimization
- Lazy loading in React
- Caching frequently accessed data

🔥 PART A: DESIGN & OOPS (PRIME FAVORITES)

✅ 1. Explain SOLID principles

SOLID is a set of **5 design principles** for maintainable code:

- **Single Responsibility** – one class, one job
- **Open/Closed** – open for extension, closed for modification
- **Liskov Substitution** – child can replace parent safely
- **Interface Segregation** – small, specific interfaces
- **Dependency Inversion** – depend on abstractions, not concrete classes

✅ 2. What is Dependency Injection?

Dependency Injection means **providing dependencies from outside** instead of creating them inside a class.

It improves **testability, loose coupling, and maintainability**.

✅ 3. Time–Space Tradeoff

We can reduce **time** by using extra **space**, or reduce **space** by spending more **time**.

Example: HashMap ($O(1)$ time, extra memory).

✅ 4. Difference between Inheritance and Composition

Inheritance = *is-a* relationship
Composition = *has-a* relationship
Composition is more flexible and preferred in large systems.

🔥 PART B: HASHING & COLLECTIONS (VERY COMMON)

✅ 5. How does HashMap work internally?

HashMap uses a **hash function** to map keys to buckets.
If collision occurs, entries are stored using **linked list** or **tree (after Java 8)**.
Average time complexity: **O(1)**.

✅ 6. How is collision handled in HashMap?

By:

- **Chaining** (linked list / tree)
 - **Rehashing** when load factor exceeds threshold
-

✅ 7. Difference between HashMap and Hashtable

HashMap	Hashtable
Not synchronized	Synchronized
Faster	Slower
Allows one null key	No null key

🔥 PART C: JVM & MEMORY (PRIME LOVES THIS)

✅ 8. Explain JVM architecture

JVM consists of:

- Class Loader
- Runtime Data Areas (Heap, Stack, Method Area)
- Execution Engine
- Garbage Collector

It enables **platform independence**.

✅ 9. What is Garbage Collection?

Garbage Collection automatically frees memory by deleting **unused objects**, preventing memory leaks.

✓ 10. Difference between Stack and Heap

Stack stores method calls and local variables.
Heap stores objects and instance variables.

🔥 PART D: DATABASE & CONSISTENCY

✓ 11. What is indexing and how does it work internally?

Indexing uses **B-Tree** or **Hash index** to reduce search time.
Instead of scanning the full table, DB jumps directly to required data.

✓ 12. Explain ACID properties

Atomicity, Consistency, Isolation, Durability
They ensure **reliable transactions**.

✓ 13. What is CAP theorem?

In a distributed system, only **two of three** can be guaranteed:

- Consistency
 - Availability
 - Partition tolerance
-

🔥 PART E: OS & CONCURRENCY

✓ 14. Explain deadlock and its prevention

Deadlock occurs when processes wait forever for resources.
Prevention:

- Avoid circular wait
 - Resource ordering
 - Preemption
-

✓ 15. Process vs Thread

Process has its own memory.
Thread shares memory within process → faster and lightweight.

🔥 PART F: SYSTEM DESIGN (VERY HIGH WEIGHT)

✅ 16. What is microservices architecture?

Application is split into **small independent services** that communicate via APIs.
Improves scalability and fault isolation.

✅ 17. Difference between REST and SOAP

REST	SOAP
Lightweight	Heavy
JSON	XML
Faster	Slower

✅ 18. What is caching?

Caching stores frequently accessed data in **fast memory** to reduce latency.
Example: Redis.

✅ 19. Explain load balancing

Load balancing distributes traffic across multiple servers to avoid overload and improve availability.

✅ 20. What is scalability?

Ability of a system to handle increased load efficiently.

Types:

- Vertical
 - Horizontal
-

✅ 21. How will you design a scalable system?

By:

- Load balancing
- Caching
- Database indexing
- Horizontal scaling

- Stateless services
-

✓ 22. Design a simple URL shortener (VERY COMMON)

Steps:

- Generate unique short ID
 - Store mapping in DB
 - Redirect using lookup
 - Add caching for popular URLs
-

✓ 23. How will you handle large-scale data?

Using:

- Sharding
 - Indexing
 - Caching
 - Distributed databases
-

🔥 PART G: FINAL HR-TECH MIX (PRIME TRAPS)

✓ 24. Why Prime role needs system design?

Prime engineers work on **large-scale, high-impact systems**, so understanding design is crucial.

✓ 25. What matters more: clean code or fast code?

Clean code first, then optimize where required.

✓ 26. How do you debug production issues?

By checking logs, reproducing issue, isolating root cause, and applying minimal fix.

✓ 27. How do you handle pressure?

By prioritizing tasks, staying calm, and breaking problems into smaller parts.